

Jak sprawdzimy, czy rozproszenie się opłaca

Plan pomiarów wytłumaczony od podstaw — razem z tym, co robią operacje genomiczne

Miłosz Kowalewski

29 września 2026

Wstęp — gdzie jesteśmy i o co chodzi teraz

Do tej pory praca odpowiadała na pytanie „**czy się da?**”. Odpowiedź brzmi: tak. Pięć operacji z biblioteki polars-bio działa na dwóch silnikach rozproszonych — Apache Ballista i Sail — bez przerabiania samych silników. To trochę jak zbudowanie samochodu i sprawdzenie, że jeździ.

Teraz przychodzi pytanie ważniejsze dla części naukowej pracy: „**czy to się opłaca, i jak bardzo?**” — czyli ile ten samochód pali, jak szybko jedzie, i czy nie lepiej byłoby po prostu pojechać autobusem. Żeby na to odpowiedzieć uczciwie, trzeba najpierw ustalić **jak mierzyć**. Temu była poświęcona sesja projektowa, którą ten dokument podsumowuje.

Dokument zakłada zero wcześniejszej wiedzy. Zaczyna od tego, czym w ogóle są dane genomiczne, po co się na nich liczy i co robi każda z pięciu operacji (z obrazkami), potem tłumaczy, jak liczy się na wielu komputerach, a na końcu — co i jak zmierzymy oraz w jakiej kolejności to zrobimy.

Wersja techniczna, ze wszystkimi szczegółami i odwołaniami do kodu, jest w podsumowanie_brainstorming_formalna.specyfikacja — w docs/superpowers/specs/2026-09-29-metodyka-benchmarkow-design.md.

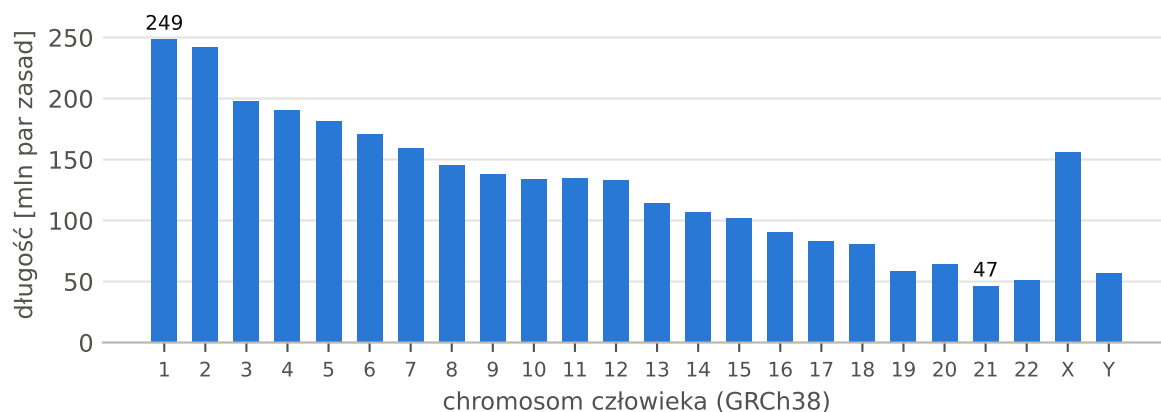
Rozdział 1 — Dane genomiczne w pigułce

DNA, chromosomy i pozycje

DNA można sobie wyobrazić jako bardzo długi tekst zapisany czterema literami: **A, C, G, T** (to cztery rodzaje „cegiełek” DNA; litera U pojawia się dopiero w RNA — roboczej kopii DNA — w miejsce T). U człowieka ten tekst ma około **3,1 miliarda liter** (liter DNA mówi się „pary zasad”, w skrócie pz). Nie jest to jeden ciągły napis — jest pocięty na **chromosomy**. Człowiek ma 24 różne chromosomy: ponumerowane od 1 do 22 oraz X i Y. **Każdy chromosom to jeden bardzo długi napis** — średnio ok. 129 milionów liter (3,1 mld / 24), od ok. 47 mln (chromosom 21) do ok. 249 mln (chromosom 1).

Liczba 3,1 mld dotyczy jednego kompletu chromosomów. W komórce człowieka są dwie kopie chromosomów 1-22 (od matki i od ojca) plus para X/X albo X/Y — ale wszystkie współrzędne opisuje się względem jednego, wspólnego komputerowego „wzorca”: **genomu referencyjnego** (obecna wersja nazywa się GRCh38). Dzięki temu „chromosom 1, pozycja 1 000 000” oznacza u każdego to samo miejsce.

Każde miejsce w genomie ma więc „adres”: **nazwa chromosomu + numer pozycji**, np. „chromosom 1, pozycja 1 000 000”. Chromosomy mają bardzo różne długości — to będzie ważne później, bo przy dzieleniu pracy między komputery chromosomy stają się „paczkami” o nierównej wadze:



Rysunek 1: Długości chromosomów człowieka. Chromosom 1 jest ponad pięć razy dłuższy od chromosomu 21.

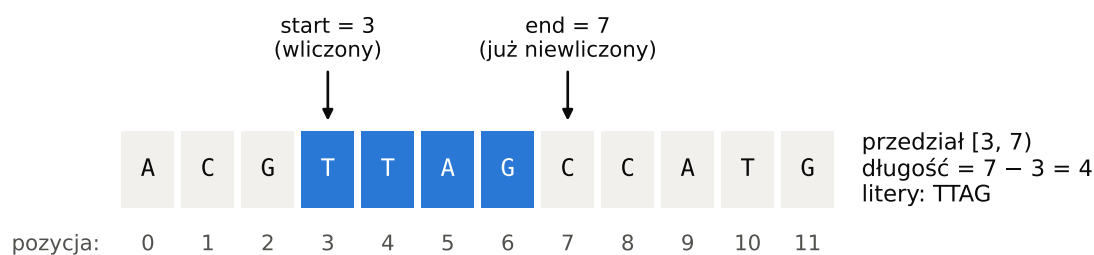
Przedział — podstawowy „klocek” danych

Większość danych genomicznych to nie pojedyncze litery, tylko **odcinki** genomu: gen, fragment genu, miejsce, w którym coś się dzieje. Taki odcinek opisuje się trzema wartościami:

(chromosom, początek, koniec) — np. (chr1, 5, 20)

i nazywa **przedziałem** albo **interwałem**. Pliki z takimi danymi (np. w formacie BED) to po prostu tabele — w każdym wierszu jeden przedział — i potrafią mieć **dziesiątki milionów wierszy**.

Jest jedna konwencja, którą warto zrozumieć raz a dobrze, bo bez niej obrazki niżej byłyby mylące. Przedziały zapisuje się jako **[początek, koniec)** — nawias kwadratowy oznacza „ta pozycja się liczy”, okrągły — „ta już nie”:



Rysunek 2: Zbliżenie na 12 pozycji przykładowego chromosomu (litery wymyślone — prawdziwy ma ich miliony). Przedział [3, 7) obejmuje pozycje 3, 4, 5 i 6, czyli litery TTAG; koniec (7) jest pierwszą pozycją poza przedziałem.

To dokładnie ta sama konwencja co wycinki napisów w Pythonie: przedział (chr1, 3, 7) to chr1[3:7], czyli litery TTAG. Dzięki temu długość to zawsze po prostu **koniec - początek**, a przedziały [5, 20) i [20, 30) **styka się, ale nie nakładają** (pozycja 20 należy tylko do drugiego).

Ważne: przedział nie zawiera liter. To tylko adres fragmentu — trzy liczby. Litery leżą w genomie referencyjnym, a przedział mówi, gdzie ich szukać. Gen o długości 80 tysięcy liter to w pliku z przedziałami jeden wiersz z trzema liczbami.

Po co to wszystko — skąd biorą się tabele przedziałów

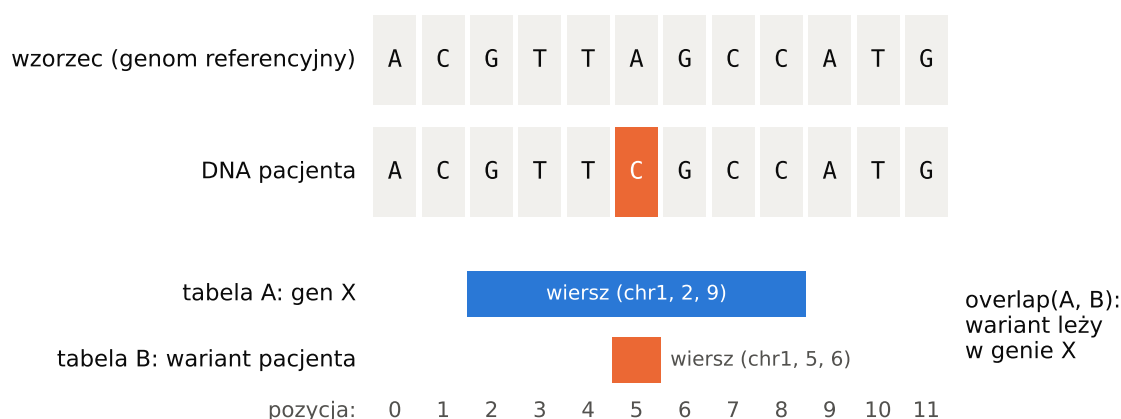
Gen to przepis na białko — a białka wykonują w organizmie prawie całą pracę: budują tkanki, trawiają, przenoszą tlen, naprawiają DNA. Jeśli w przepisie jest „literówka” (**mutacja**, fachowo **wariant**), białko może wyjść wadliwe, a to może oznaczać chorobę. Przykładowo mutacje w genie BRCA1 znacząco zwiększają ryzyko raka piersi i jajnika, a mutacje w genie CFTR powodują mukowiscydozę.

Każdy człowiek różni się od genomu referencyjnego w **kilku milionach** miejsc. Prawie wszystkie te różnice są nieszkodliwe (to one sprawiają m.in., że ludzie różnią się wyglądem). Pytanie lekarza brzmi więc: „**które z tych milionów różnic trafiły w geny — i w które?**”. Tylko te warto oglądać dokładnie.

Operacje na przedziałach **nie porównują chromosomu z chromosomem ani dwóch ludzi ze sobą**. Porównują **dwie tabele** pochodzące z różnych źródeł, opisujące te same chromosomy:

	Co to jest	Skąd
genom referencyjny	wspólny wzorec — ustalony napis ~3,1 mld liter	jeden dla wszystkich
katalog genów	gdzie na wzorcu leży każdy z ~20 tys. genów (u wszystkich ludzi w tym samym miejscu)	wiedza naukowa, publiczne bazy
warianty pacjenta	miejsca, w których DNA tej jednej osoby różni się od wzorca	sekwencjonowanie tej osoby
odczyty z sekwenatora	miliony krótkich (~150 liter) kawałków DNA osoby, „przyłożonych” do wzorca	sekwencjonowanie tej osoby
czarna lista	tam, gdzie pasują regiony, w których sekwenatory notorycznie się mylą	publiczne bazy

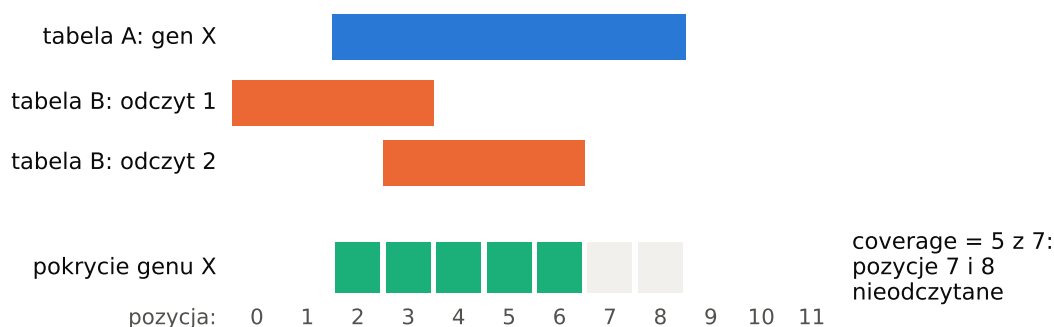
Oto cała idea na jednym obrazku. Porównujemy DNA pacjenta ze wzorcem, na pozycji 5 wychodzi różnica — zapisujemy ją jako wiersz tabeli wariantów. Katalog genów mówi, że gen X zajmuje pozycje 2-8. Operacja **overlap** stwierdza, że wariant leży w genie X — i to jest informacja, która interesuje lekarza:



Rysunek 3: Skąd biorą się tabele: katalog genów (tabela A) i warianty pacjenta (tabela B) to dwie listy adresów na tym samym chromosomie.

Podobnie z **coverage**. Sekwenator nie czyta chromosomu w całości, tylko krótkie kawałki — każdy odczytany kawałek to znowu wiersz z adresem. Jeśli część genu nie została

odczytana, na tych pozycjach nie wiemy, czy pacjent ma mutację — wynik badania jest tam niepewny:



Rysunek 4: Sens coverage: gen X ma 7 pozycji, odczyty pokrywają 5 z nich; pozycje 7 i 8 pozostały nieodczytane.

Porównuje się zawsze wiersze **z tego samego chromosomu** — gen z chromosomu 1 nigdy nie „spotka” wariantu z chromosomu 2. To nie przeszkoda, tylko zaleta: właśnie dzięki temu pracę da się podzielić między komputery według chromosomów (rozdział 3).

Skąd duże dane? Jeden pacjent to miliony wariantów i dziesiątki milionów odczytów. Badania naukowe porównują jednak **tysiące lub setki tysięcy osób** (tzw. kohorty), chorych i zdrowych, żeby odkryć, w których genach mutacje częściej występują u chorych. Każda osoba ma własną tabelę — i dopiero wtedy tabelę mają miliardy wierszy i przestają mieścić się na jednym komputerze.

Przykład, na którym pokażę wszystkie operacje

Weźmy dwie małe tabele przedziałów na chromosomie 1:

- **A** = [5, 20), [15, 30), [40, 55), [70, 80)
- **B** = [10, 25), [45, 50), [52, 65), [88, 95)

A i B to tylko nazwy dwóch tabel — nie mają nic wspólnego z literą A w DNA. W prawdziwym zastosowaniu A mogłaby być np. katalogiem genów, a B — wariantami pacjenta.

Na obrazkach A jest **niebieska**, B **pomarańczowa**, a wynik operacji **zielona**. Gdy dwa przedziały z tej samej tabeli się nakładają (jak [5, 20) i [15, 30) w A), rysuję je w dwóch rzędach, żeby się nie zasłaniały — tak samo robią przeglądarki genomu.

Ważne: obrazki nie są rysowane z pamięci. Skrypt, który je tworzy, wywołuje prawdziwe polars-bio na tych danych i rysuje dokładnie to, co biblioteka zwróciła; dodatkowo sprawdza, czy narysowane kawałki zgadzają się z liczbami z biblioteki.

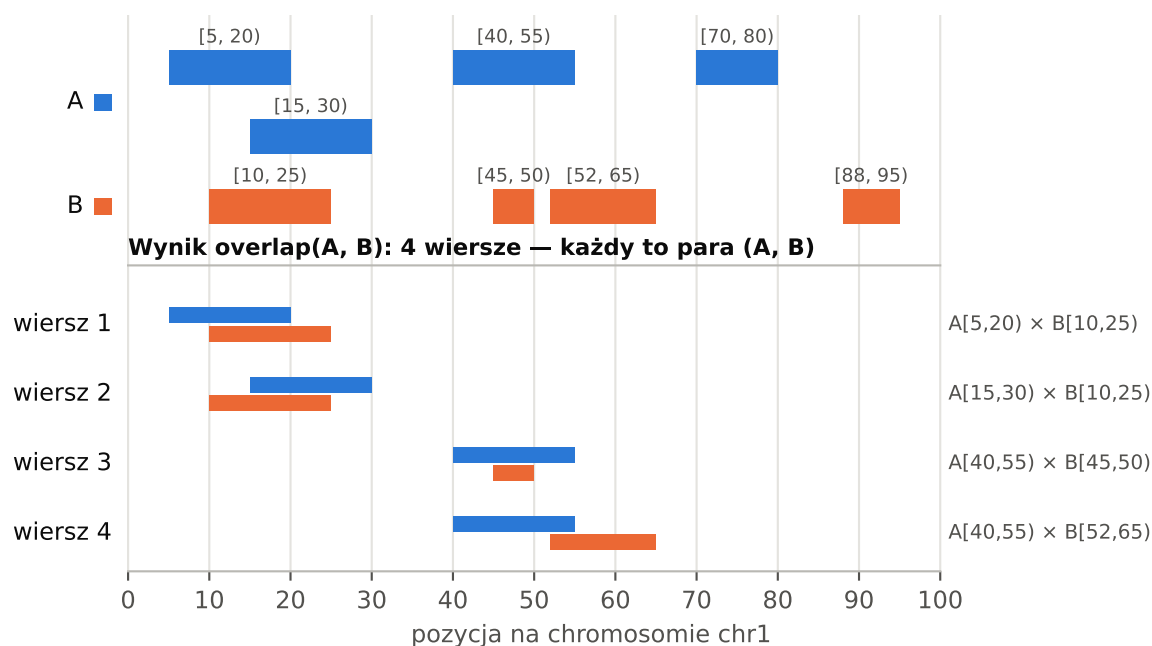
Rozdział 2 — Pięć operacji, każda na obrazku

overlap — „które pary się nakładają?”

Pytanie: dla każdego przedziału z A znajdź **wszystkie** przedziały z B, z którymi ma choć jedną wspólną pozycję.

Przykład z życia: mamy katalog genów (A) i warianty znalezione u pacjenta (B) — które warianty leżą wewnątrz których genów? (To przykład z poprzedniego rozdziału.)

Jak czytać: wynik to **pary**, nie nowe przedziały. [40, 55) z A pojawia się w wyniku **dwa razy**, bo nakłada się z dwoma przedziałami z B. [70, 80) nie pojawia się wcale — nic z B go nie



Rysunek 5: $\text{overlap}(A, B)$. Każdy wiersz wyniku to para: przedział z A (u góry) i przedział z B (u dołu).

dotyka. Przy dużych danych wynik bywa ogromny: dla par zbiorów z publikacji polars-bio sięga setek milionów wierszy.

nearest — „co jest najbliżej?”

Pytanie: dla każdego przedziału z A znajdź **jeden najbliższy** przedział z B i podaj odległość. Przedział nakładający się ma odległość 0.

Przykład z życia: mutacja leży między genami — który gen jest najbliżej i jak daleko?

Jak czytać: trzy pierwsze przedziały A nakładają się z czymś w B, więc mają odległość 0. Dla [70, 80) najbliższy jest [52, 65); przerwa to pozycje 65, 66, 67, 68, 69 — czyli 5.

Ciekawostka, która ma znaczenie: [40, 55) nakłada się z **dwoma** przedziałami B, oba mają odległość 0 — to **remis**. Biblioteka zwróciła jeden z nich. Który — nie jest częścią „umowy” operacji. Dlatego przy sprawdzaniu poprawności wyników porównujemy **odległości**, a nie to, którego sąsiada wybrano; inaczej zgłaszałibyśmy błędy tam, gdzie ich nie ma.

coverage — „ile mojego przedziału jest przykryte?”

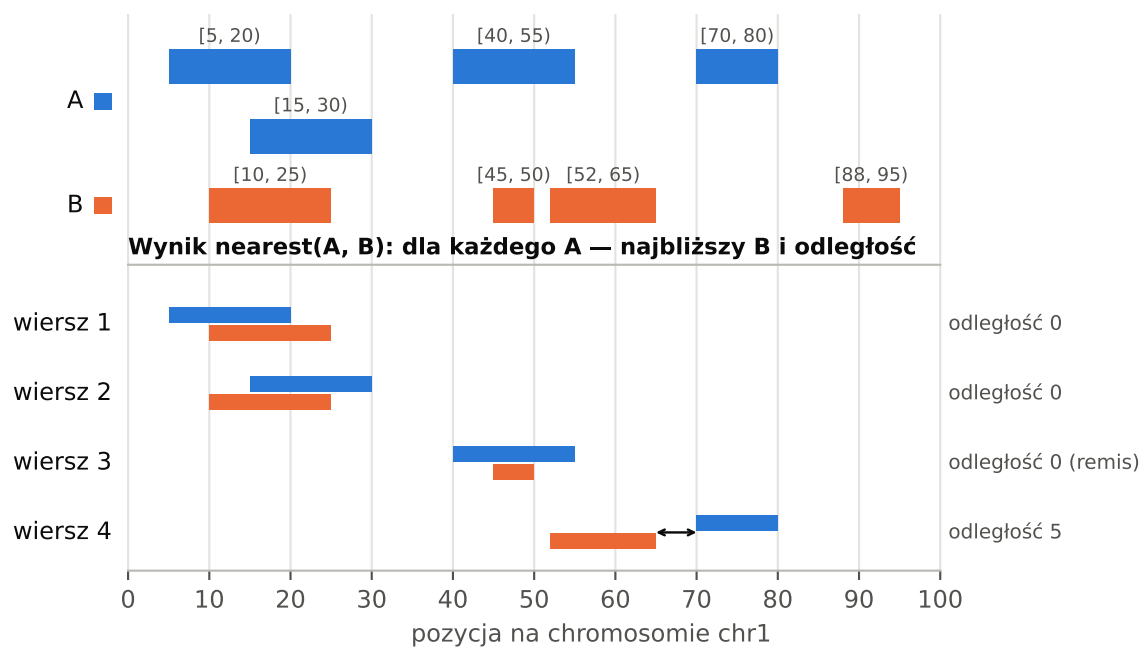
Pytanie: dla każdego przedziału z A policz, **ile jego pozycji** przykrywa B.

Przykład z życia: mamy fragmenty genów (A) i obszary, które udało się odczytać w badaniu (B) — jaka część każdego fragmentu została faktycznie zbadana?

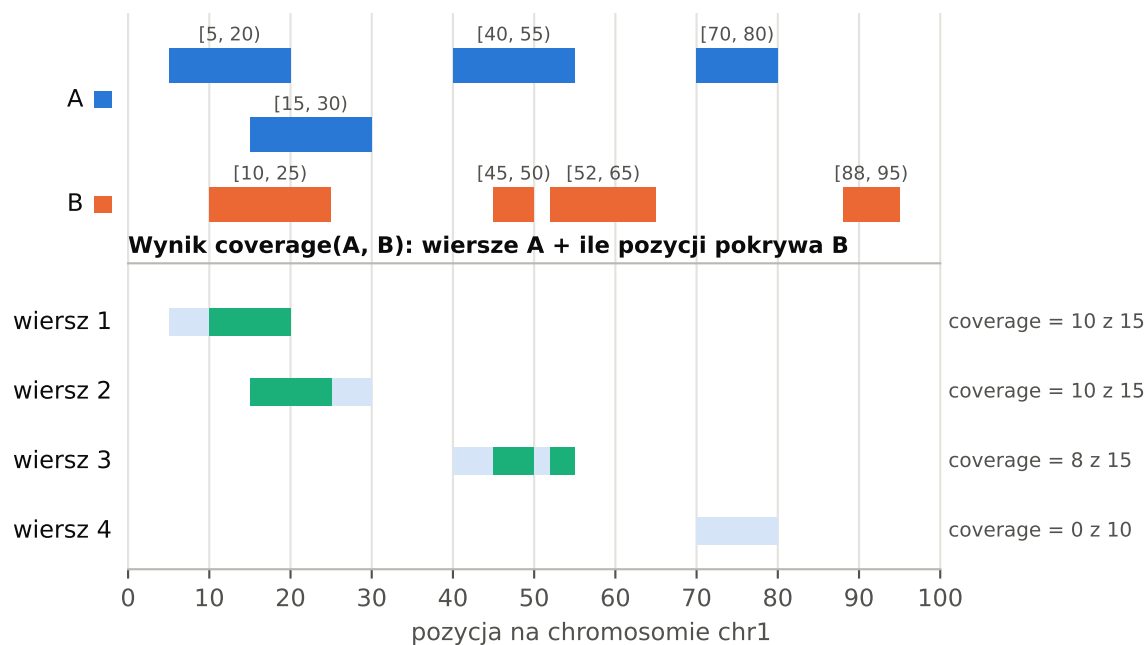
Jak czytać: [40, 55) jest przykryty w dwóch miejscach: przez [45, 50) (5 pozycji) i przez [52, 65), który sięga tylko do 55 (3 pozycje) — razem 8. [70, 80) ma pokrycie 0. Wynik ma tyle wierszy, ile A — operacja nie tworzy par, tylko dopisuje liczbę do każdego wiersza A.

merge — „sklej to, co się nakłada”

Pytanie: połącz nakładające się przedziały **jednej** tabeli w ciągłe odcinki.

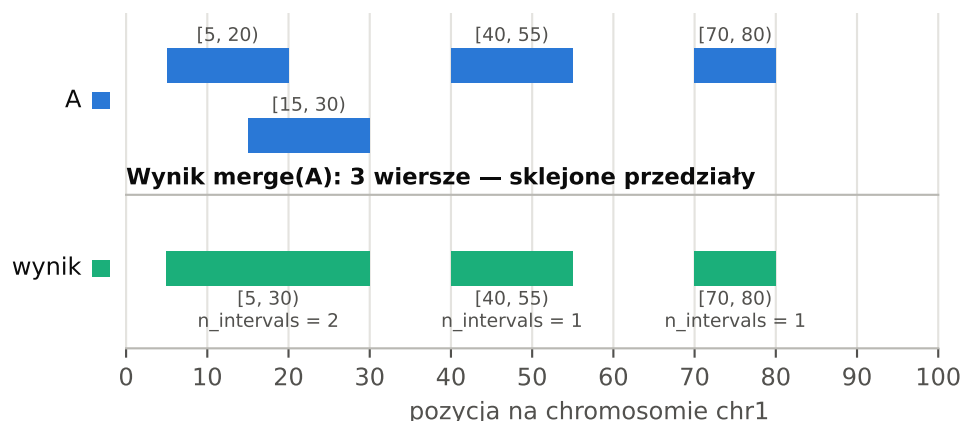


Rysunek 6: nearest(A, B). Każdy wiersz: przedział z A, jego najbliższy sąsiad z B i odległość; strzałka pokazuje przerwę.



Rysunek 7: coverage(A, B). Wynik to wiersze A; zielone są pozycje przykryte przez B, jasne — nieprzykryte.

Przykład z życia: różne źródła opisały te same regiony trochę inaczej, z zakładkami — chcemy jednej, uproszczonej listy obszarów.



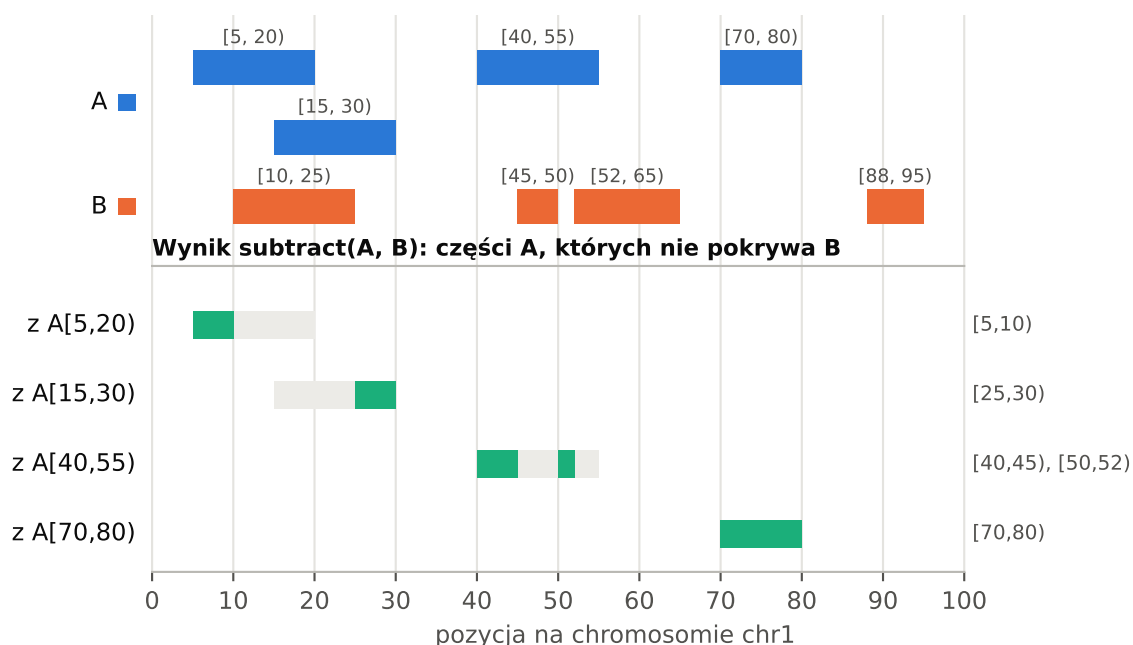
Rysunek 8: merge(A). [5, 20) i [15, 30) zlewają się w [5, 30); kolumna n_intervals mówi, ile przedziałów sklejono.

Jak czytać: z czterech przedziałów robią się trzy. n_intervals = 2 przy [5, 30) mówi, że powstał ze sklejenia dwóch. To jedna z pięciu operacji, która działa na jednej tabeli.

subtract — „odejmij”

Pytanie: z każdego przedziału A wytnij te pozycje, które przykrywa B; zostaw resztę.

Przykład z życia: istnieją listy regionów genomu, które notorycznie dają błędne wyniki (tzw. czarne listy) — przed analizą wycina się je z interesujących nas obszarów.



Rysunek 9: subtract(A, B). Szare tło to pierwotny przedział A, zielone — to, co z niego zostało po odjęciu B.

Jak czytać: odejmowanie działa **osobno dla każdego przedziału A**. [40, 55) rozpada się

na **dwa** kawałki, [40, 45) i [50, 52), bo B wycina ze środka [45, 50), a z końca wszystko od 52. [70, 80) zostaje w całości, bo nic z B go nie dotyka.

Podsumowanie operacji

Operacja	Pytanie	Wejście	Wynik
overlap	które pary się nakładają?	A, B	pary (A, B)
nearest	co jest najbliższe?	A, B	każde A + najbliższy B + odległość
coverage	ile A przykrywa B?	A, B	każde A + liczba pozycji
merge	sklej nakładające się	A	sklejone przedziały + ile ich sklejono
subtract	wytnij B z A	A, B	kawałki A

Gdzie te operacje pracują w prawdziwej analizie

W badaniach kohortowych operacje na przedziałach są w dużej mierze **przygotowaniem danych** (i tworzeniem cech) przed właściwą analizą. Typowy potok wygląda w uproszczeniu tak:

1. **sekwenator** produkuje surowe odczyty — krótkie kawałki DNA każdej osoby;
2. **przyłożenie odczytów do wzorca** — każdy odczyt dostaje adres (chromosom, początek, koniec);
3. **wykrycie wariantów** — dla każdej osoby powstaje tabela różnic względem wzorca;
4. **kontrola jakości** — *coverage* (czy region był dobrze odczytany), *subtract* (wycięcie regionów z czarnej listy);
5. **adnotacja** — *overlap* (w którym genie leży wariant?), *nearest* (który gen jest najbliższy?);
6. **agregacja do cech** — *overlap* + grupowanie, np. „ile rzadkich, groźnych mutacji ma osoba X w genie Y”; powstaje tabela osoby × geny;
7. **analiza** — statystyka albo uczenie maszynowe, z etykietami typu „chory / zdrowy”.

Na końcu częściej stoi statystyka niż uczenie maszynowe. Klasyczne narzędzia to:

- **GWAS** (badanie asocjacji całogenomowej) — dla każdego wariantu osobno sprawdza się statystycznie, czy występuje częściej u chorych; to miliony testów naraz, więc wymagają specjalnej korekty, żeby nie „odkrywać” przypadkowych zależności;
- **testy agregujące po genach** — zbierają wszystkie rzadkie warianty w danym genie i badają gen jako całość (tu właśnie potrzebny jest *overlap*, żeby przypisać warianty do genów);
- **poligeniczne skale ryzyka** — ważona suma tysięcy wariantów dająca jedną liczbę „ryzyka genetycznego” danej osoby.

Dlaczego nie „po prostu ML”? Z punktu widzenia uczenia maszynowego to trudny przypadek: cech (wariantów) są miliony, a osób — tysiące; pochodzenie etniczne wpływa jednocześnie na warianty i na częstość chorób, więc naiwny model łatwo znajduje fałszywe zależności; a w medycynie liczy się wyjaśnialność („gen X, efekt taki a taki, z taką pewnością”). Uczenie maszynowe, w tym głębokie sieci, świetnie sprawdza się bliżej biologii — np. w przewidywaniu, czy konkretna mutacja szkodzi białku (znane modele: AlphaMissense, SpliceAI). One też potrzebują danych przygotowanych operacjami na przedziałach.

Nie wszystko jest jednak tylko przygotowaniem: część analiz **sama jest** operacją na przedziałach (np. czy warianty związane z chorobą częściej niż przypadkiem leżą w regionach sterujących pracą genów — to *overlap* plus test statystyczny), a *coverage* bywa w diagnostyce wynikiem końcowym („ten gen odczytano dostatecznie dokładnie — wynikowi można ufać”).

Co to znaczy dla tej pracy: przy kohortach rzędu 100 tysięcy osób właśnie warstwa przygotowania danych staje się wąskim gardłem — to praca na miliardach wierszy, często powtarzana przy każdej zmianie katalogu genów czy filtrów jakości. Praca nie dotyczy

modelu, który wyciąga wnioski o chorobach, tylko **silnika**, który w rozsądnym czasie **dostarcza mu danych**. Rozproszenie ma sens dokładnie tutaj.

Rozdział 3 — Jak liczy się na wielu komputerach (bez żargonu)

Proces, węzeł, executor, scheduler

Proces to uruchomiony program z **własną pamięcią**. Dwa procesy nie widzą nawzajem swojej pamięci — jeśli jeden ma coś przekazać drugiemu, musi to „wysłać”, tak jak list. To jest sedno całej trudności w obliczeniach rozproszonych.

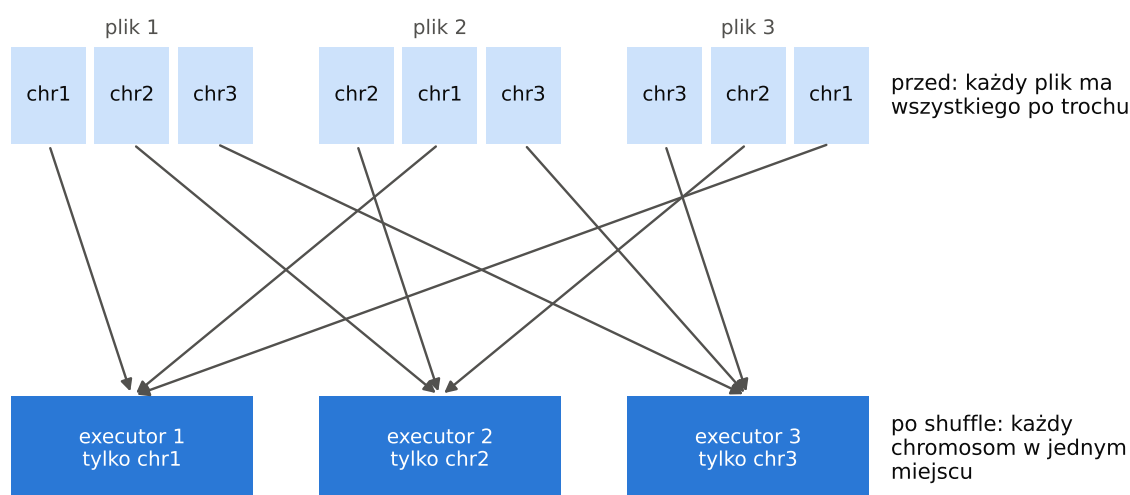
Dobra analogia to **kuchnia restauracji**:

- **klient** (nasz program) zamawia danie, czyli wysyła **zapytanie**;
- **scheduler** to szef kuchni — nie gotuje sam, tylko rozdziela zadania;
- **executory** to kucharze; każdy stoi przy swoim stanowisku, czyli **węźle** (komputerze albo jego kawałku);
- **plan zapytania** to przepis. Żeby szef mógł podać przepis kucharzowi, przepis musi dać się **zapisać na kartce** (to się nazywa serializacja). Nasze operacje genomiczne to „niestandardowe dania”, których silnik nie zna — dlatego wcześniej trzeba było napisać własnych „tłumaczy” (kodery), którzy umieją je zapisać i odczytać.

Dwa sposoby dzielenia pracy

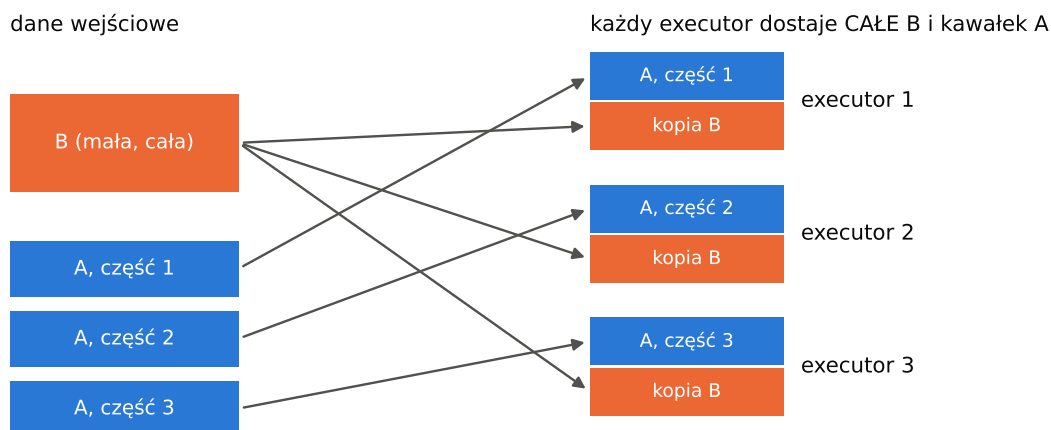
Operacje rozpraszają się na dwa sposoby — i nie jest to nasz wybór, tylko wynika z tego, co każda operacja musi „widzieć”.

Przetasowanie po chromosomie (shuffle) — dla merge i subtract. Te operacje potrzebują widzieć **cały chromosom naraz** (żeby skleić sąsiadów, trzeba ich mieć obok siebie), ale różne chromosomy są od siebie niezależne. Więc dane się tasuje tak, żeby każdy kucharz dostał wszystkie kawałki swojego chromosomu:



Rysunek 10: Shuffle: na starcie każdy plik ma wszystkiego po trochu; po przetasowaniu każdy chromosom trafia w jedno miejsce.

Rozesłanie kopii (broadcast) — dla nearest i coverage. Tu każdy przedział A musi móc zajrzeć do **całej** tabeli B (najbliższy sąsiad może być gdziekolwiek). Więc mała tabela B jest kopiowana do każdego kucharza, a duża tabela A dzielona na kawałki:



Rysunek 11: Broadcast: każdy executor dostaje całą tabelę B i swój kawałek A.

Pułapka przetasowania: skoro jeden chromosom trafia w całości do jednego kucharza, to kucharzy z pracą może być najwyżej tyle, ile chromosomów (~24). A że chromosomy są nierówne (chromosom 1 jest ponad 5 razy większy od 21 — obrazek w rozdziale 1), przy wielu kucharzach wszyscy czekają na tego, który dostał chromosom 1. Na laptopie, gdzie będziemy mieć najwyżej 3 „kucharzy”, tego nie zobaczymy — ale w chmurze tak, i warto to w pracy opisać.

Czy to naprawdę jest rozproszone? (pytanie P0)

Do tej pory Ballista była uruchamiana w trybie, w którym **szef kuchni i kucharz to jedna osoba**, tzn. jeden proces. Przepis był naprawdę zapisywany na kartce i odczytywany, dane naprawdę tasowane — ale wszystko w jednej głowie. To mocna przesłanka, że rozproszenie działa, ale jeszcze nie dowód.

Dlatego pierwszym zadaniem (P0) jest uruchomienie **szefa i dwóch kucharzy jako trzech osobnych programów** i sprawdzenie czterech rzeczy:

1. to naprawdę osobne programy (każdy ma inny numer w systemie);
2. wynik jest poprawny — taki sam jak z polars-bio na jednym komputerze;
3. **obaj** kucharze coś ugotowali, a dane przeszły między nimi (a nie wszystko zrobił jeden);
4. jeśli jednemu kucharzowi **zabierzemy tłumacza** (kodery), zapytanie musi się wywrócić — to dowodzi, że kucharz naprawdę sam czyta przepis.

W P0 **nie mierzymy czasów** — pytanie brzmi „czy”, nie „jak szybko”, a czasy na pięciu przedziałach niczego by nie znaczyły.

Sail ma inny kłopot. Lokalnie działa jako jeden program, a do tego ma „kłódkę”. Skąd kłódka? polars-bio trzyma swoje robocze tabele w jednym, wspólnym notesie. Gdy dwóch kucharzy pisze w nim naraz, nadpisują sobie notatki i część wyników cicho znika (to zostało wykryte i wyjaśnione na wcześniejszym etapie). Kłódka sprawia, że pisze tylko jeden naraz — wyniki są poprawne, ale na laptopie polars-bio w Sailu liczy **po kolei**, nie równolegle.

Żeby Sail działał naprawdę jako osobne programy, potrzebuje **Kubernetesa** — systemu, który uruchamia programy w tzw. kontenerach (odizolowanych „pudełkach”) i rozkłada je po maszynach. Na laptopie z małą ilością pamięci poprzednia próba się nie zmieściła; będzie druga, ograniczona czasowo próba po zwiększeniu pamięci. Jeśli się nie uda — to też jest wynik, który opisujemy.

Rozdział 4 — Co i jak zmierzmy

Pytania, na które mają odpowiedzieć pomiary

	Pytanie po ludzku
P0	Czy Ballista naprawdę dzieli pracę między osobne programy? (warunek wstępny)
P1	Czy wersja rozproszona jest szybsza od zwykłego polars-bio? O ile? Dla których operacji i od jakiej wielkości danych?
P2	Co się dzieje, gdy dokładamy kolejne węzły: $1 \rightarrow 2 \rightarrow 3$?
P3	Czy „najlepszy” algorytm wyszukiwania przedziałów zostaje najlepszy po rozproszeniu? (pytanie poboczne)

Porównanie **Ballista kontra Sail** nie jest osobnym eksperymentem — wychodzi samo, bo oba silniki porównujemy z tym samym punktem odniesienia. Dochodzą do tego różnice, których nie widać w czasach, a które też są wynikiem: ile pracy wymagała integracja, czego każdy silnik potrzebuje do działania (Sail — Kubernetesa) i jakie ma ograniczenia.

Z czym porównujemy — i dlaczego to ważne

Wyobraźmy sobie przeprowadzkę. Mamy trzy małe ciężarówki (klaster z trzech węzłów) i chcemy wiedzieć, czy to dobry pomysł. Można porównać je z **jedną małą ciężarówką** — i trzy wypadną świetnie. Ale uczciwie jest też porównanie z **jedną dużą ciężarówką o tej samej ładowności co trzy małe razem** — bo może się okazać, że jedna duża jest szybsza, bo nie trzeba koordynować trzech kierowców.

Znany artykuł „*Scalability! But at what COST?*” (McSherry i in., 2015) pokazał, że wiele systemów rozproszonych chwali się pięknym przyspieszeniem względem jednej małej ciężarówki, a przegrywa z jedną porządną. Dlatego mierzymy **oba** punkty odniesienia:

- **wariant A** — polars-bio na zasobach jednego węzła (jedna mała ciężarówka),
- **wariant B** — polars-bio na zasobach całego klastra (jedna duża ciężarówka).

Czego się spodziewamy

Hipotezy — czyli zakłady, które pomiary sprawdzą (a przegrany zakład też jest wartościowym wynikiem):

- **dla małych danych rozproszenie przegra** — koszt „organizacji kuchni” (zapisywanie przepisu, tasowanie danych) będzie większy niż zysk z podziału pracy;
- **operacje z tasowaniem skalują się lepiej niż te z rozsyłaniem kopii** — przy rozsyłaniu każdy kucharz powtarza tę samą pracę przygotowawczą na swojej kopii B;
- **po rozproszeniu ranking algorytmów może się zmienić** — w polars-bio przed wyszukiwaniem buduje się „indeks” (coś jak spis treści) jednej z tabel. Na jednym komputerze buduje się go raz; po rozproszeniu **każdy kucharz buduje swój**. Algorytm, którego indeks buduje się taniej, może więc wygrać po rozproszeniu, choć przegrywa lokalnie.

Jak zrobimy „klaster” na laptopie

Procesor laptopa ma **6 rdzeni** (rdzeń to jakby osobny „mózg” do liczenia), a każdy rdzeń obsługuje **2 wątki** naraz. Umawiamy się, że **jeden węzeł = jeden rdzeń**, i każdemu programowi „przypinamy” jego rdzeń (narzędzie taskset), żeby nie podbierał mocy innym:



Rysunek 12: Przydział rdzeni: rdzeń 0 dla systemu i programów pomocniczych, rdzenie 1–3 jako trzy węzły, dwa ostatnie wolne.

Dzięki temu dołożenie węzła naprawdę oznacza dołożenie mocy obliczeniowej. To nie jest prawdziwy klaster — węzły dzielą pamięć i dysk, nie ma między nimi prawdziwej sieci — i te ograniczenia uczciwie opisujemy w pracy. Ale pomiary lokalne mają być **pełnoprawnym wynikiem pracy**: praca musi się obronić także wtedy, gdy chmury nie będzie. Jeśli chmura się pojawi, te same pomiary powtórzymy w większej skali tymi samymi narzędziami.

Dane

Nie wymyślamy własnych danych — bierzemy **te same zbiory, których użyto w publikacji polars-bio**. To prawdziwe dane z publicznych baz genomycznych, m.in. fragmenty genów (eksony) i tzw. łańcuchy dopasowań — odcinki genomu człowieka dopasowane do genomów innych gatunków (szczura, dziobaka, alpaki, żaby, oposa). Zbiory mają od 199 tysięcy do 128 milionów przedziałów. Dzięki temu nasze wyniki da się zestawić z opublikowanymi.

Na laptopie zmieszczą się tylko mniejsze z nich. Zamiast zgadywać które, stosujemy prostą regułę: każdą parę najpierw liczymy zwykłym polars-bio i mierzymy, ile pamięci zużył; bierzemy tylko te, które zajmują najwyżej **jedną trzecią** wolnej pamięci (zapas jest potrzebny, bo przy rozsyłaniu kopii każdy z trzech węzłów trzyma własną kopię B). Reszta czeka na chmurę.

Dane zapisujemy w formacie **Parquet** zamiast CSV. CSV to zwykły tekst, który komputer musi żmudnie „przeczytać”; Parquet to format przygotowany do szybkiego wczytywania — przy CSV mierzylibyśmy głównie czytanie tekstu, a nie same operacje.

Limit 16 MB. Przy rozsyłaniu kopii Ballista dołącza całą tabelę B **do przepisu**, a przepis ma ograniczony rozmiar — domyślnie 16 MB, czyli około pół miliona przedziałów. Sprawdziłem w kodzie Ballisty, że ten limit **da się podnieść zwykłym ustawieniem**, bez przerabiania silnika. Haczyk: przepis trafia do każdego zadania osobno, więc duża tabela B jest wysyłana wielokrotnie. Zadziała, ale będzie to kosztować — i to też jest wynik do opisanania.

Co mierzymy

- **Czas** — od wysłania zapytania do otrzymania ostatniego wiersza wyniku.
- **Pamięć** — największa ilość pamięci, jaką zajął każdy z programów.
- **Ile danych przetasowano** — ile przeszło między węzłami (tłumaczy, skąd bierze się narzut).
- **Rozbicie czasu na etapy** — ile trwało czytanie, tasowanie, a ile samo liczenie; w Sailu dodatkowo, ile czasu programy **czekały na kłódkę**. To pokaże liczbowo, ile kosztuje wspólny notes polars-bio.

Jak mierzymy uczciwie

Pomiar czasu na komputerze jest zaskakująco kapryśny. Dlatego:

- **najpierw jeden przebieg „na rozgrzewkę”**, który wyrzucamy — pierwszy raz zawsze trwa dłużej (programy dopiero się „rozkrecają”);
- potem **5 pomiarów** i bierzemy **medianę** (wartość środkową) — jeden pechowy pomiar, np. gdy Windows akurat coś robił w tle, nie zepsuje wyniku tak, jak zepsułby średnią;
- scenariusze wykonujemy w **losowej kolejności** — laptop się nagrzewa i zwalnia, więc gdyby jeden scenariusz był zawsze na końcu, zawsze trafiałby na „zmęczony” procesor;
- na początku i końcu każdej serii puszczaamy ten sam **przebieg kontrolny**; jeśli wyniki różnią się o więcej niż 10%, coś się zmieniło w trakcie i serię powtarzamy;
- pomiar, w którym komputerowi zabrakło pamięci i sięgnął po **swap** (dysk udający pamięć, setki razy wolniejszy), jest **nieważny**;
- pomiary robimy z **zamkniętym VS Code** i innymi programami, na zasilaczu — to zwalnia prawie dwa razy więcej pamięci dla eksperymentów.

Każdy pomiar sprawdza też poprawność. Przy każdym przebiegu liczymy liczbę wierszy wyniku i **sumę kontrolną** — coś jak suma kwot z paragonów: nie zależy od kolejności paragonów, ale zmienia się, gdy choć jedna kwota jest inna. Jeśli wersja rozproszona da inny wynik niż zwykle polars-bio, pomiar jest odrzucany. Nie da się więc „przyspieszyć” przez przypadkowe zgubienie danych.

Ile tego będzie

Każdą z pięciu operacji mierzymy na około trzech parach danych (od małych do największych, które mieszczą się w pamięci) — razem 16 scenariuszy. Każdy scenariusz w 9 wariantach (polars-bio, Ballista i Sail, każdy na 1, 2 i 3 węzłach), po 6 przebiegów. Do tego pytanie P3 o algorytmy. Razem około **1200 przebiegów** — rząd **kilku godzin** pracy laptopa.

Kolejność jest ułożona tak, żeby każdy etap sam w sobie coś dawał:

1. **próbny przebieg** wszystkiego na najmniejszych danych (kilka minut) — czy narzędzie działa;
2. **kalibracja** — które dane zmieszczą się w pamięci;
3. **P1** — 1 i 3 węzły: czy rozproszenie się opłaca;
4. **P2** — dołożenie 2 węzłów: pełna krzywa;
5. **P3** — algorytmy;
6. **analiza** — tabele i wykresy do pracy.

Gdyby czegoś zabrakło po kroku 3, i tak mamy odpowiedź na najważniejsze pytanie.

Rozdział 5 — Plan działania krok po kroku

A. Dokumentacja — specyfikacja, ten dokument i jego wersja techniczna; przegląd; zapis w repozytorium.

B. Plan implementacji — rozpisanie pracy na małe zadania, każde z testem.

C. Implementacja — zadanie po zadaniu, zawsze **najpierw test, potem kod** (tzw. TDD: test najpierw musi nie przechodzić, a przejść dopiero po napisaniu kodu — dzięki temu wiadomo, że testuje to, co trzeba). Po każdym zadaniu zapis w repozytorium. Postęp jest odhaczany w pliku planu, więc przerwanie pracy (limit, awaria, restart) niczego nie gubi — kolejna sesja zaczyna od pierwszego nieodhaczonego punktu. Kolejno:

1. P0 — Ballista jako osobne programy, cztery dowody;
2. pobranie danych i przejście na Parquet;
3. narzędzie pomiarowe;

4. dodatkowe pomiary w Sailu (czas kłódki) i przekazywanie wyboru algorytmu;
5. próba Saila na Kubernetesie.

D. Pomiary — uruchamiane z terminala, zawsze po liście kontrolnej (zamknięty VS Code i programy w tle, ustawienia pamięci, zasilacz): kalibracja → P1 → P2 → P3 → analiza.

Rozdział 6 — Co jest jeszcze otwarte

- który punkt odniesienia (A czy B) będzie w pracy główny — mierzymy oba, więc nic nie blokuje;
- czy będzie dostęp do chmury i kto ją sfinansuje — oszacowanie kosztów zrobimy po kalibracji, kiedy będą prawdziwe czasy lokalne;
- architektura **lakehouse** — sposób przechowywania danych w formie „inteligentnych tabel”, które wiedzą, gdzie leży który kawałek (np. każdy chromosom osobno). To mogłoby oszczędzić tasowania, bo dane od początku leżałyby tak, jak trzeba. Narzędzie jest projektowane tak, żeby dało się to dodać później bez przebudowy.

Słowniczek

Pojęcie	Znaczenie
para zasad (pz)	jedna „litera” DNA; jednostka długości w genomie
chromosom	jeden z 24 fragmentów, na które podzielony jest genom człowieka; jeden długi napis
genom referencyjny	wspólny wzorzec DNA człowieka, względem którego podaje się wszystkie pozycje (GRCh38)
gen	fragment DNA będący przepisem na białko; ok. 20 tys. u człowieka
wariant (mutacja)	miejsce, w którym DNA danej osoby różni się od wzorca
odczyt	krótki kawałek DNA odczytany przez sekwenator i przyłożony do wzorca
kohorta	duża grupa osób badanych razem (tysiące–setki tysięcy)
GWAS	badanie statystyczne: które warianty występują częściej u chorych
przedział / interwał	odcinek genomu: (chromosom, początek, koniec)
[początek, koniec)	zapis, w którym początek się liczy, a koniec już nie
BED, Parquet	formaty plików z danymi; Parquet jest szybszy do wczytania
proces	uruchomiony program z własną pamięcią
węzeł	komputer (u nas: rdzeń procesora) wykonujący część pracy
executor	program-„kucharz” liczący na węzeł
scheduler	program-„szef kuchni” rozdzielający zadania
plan zapytania	„przepis” na obliczenie
serializacja, koder	zapisanie przepisu „na kartce”, i program, który to umie
shuffle	przetasowanie danych tak, by każdy chromosom trafił w jedno miejsce

Pojęcie	Znaczenie
broadcast	rozesłanie kopii małej tabeli do wszystkich węzłów
UDTF	sposób, w jaki Sail wywołuje nasz kod (polars-bio) na grupie danych
kłódka (lock)	mechanizm pozwalający tylko jednemu programowi naraz korzystać z czegoś wspólnego
Kubernetes	system uruchamiający programy w kontenerach na wielu maszynach
rdzeń, wątek	„mózg” procesora; każdy rdzeń obsługuje 2 wątki
swap	dysk używany jako zapasowa pamięć — bardzo wolny
mediana	wartość środkowa po posortowaniu wyników
suma kontrolna	liczba „streszczająca” wynik; inna, gdy wynik jest inny
benchmark	uporządkowany, powtarzalny pomiar wydajności
lakehouse	sposób przechowywania danych jako tabel z informacją, gdzie co leży